“PageTurners Bookstore –

Data Management using SQL Server”

FACULTY OF MANAGEMENT

SIR PADAMPAT SINGHANIA UNIVERSITY

UDAIPUR – 313601, RAJASTHAN, INDIA

REPORT

On

"PageTurners Bookstore-- Data Management using SQL Server"

MASTER OF BUSINESS ADMINISTRATION

By

Student Name: Pooja Kumari Luhar

Enrolment No. 24MBA00500

Under the supervision of

Prof. Gour Saha

1. History of PageTurners Bookstore

The store was established in 2010 in a mid-sized city of Bhilwara, Rajasthan with the aim of providing a wide range of books to students, professionals, and casual readers. Over time, it has grown from a small local shop into a well-organized PageTurners Bookstore offering genres such as Fiction, Science, Technology, and Business.

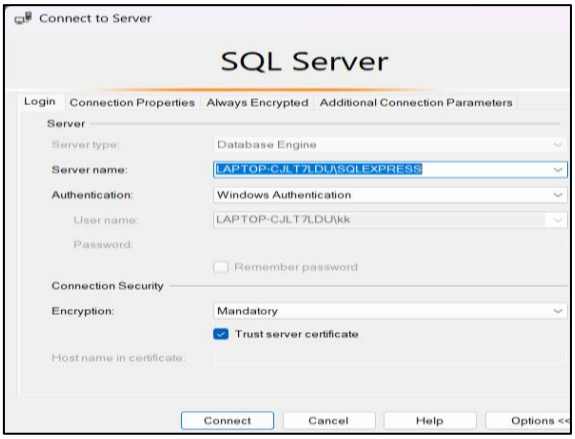
With increasing sales and customers, manual record-keeping became difficult. To solve this, the owner decided to use a database system. Implementing the PageTurners Bookstore's data in SQL Server ensures proper management of books, authors, customers, and orders, while also helping in sales analysis and business planning.

2. Background of the Study

PageTurners Bookstore handles a lot of data about books, authors, customers, orders, and suppliers. PageTurners Bookstores keep these records manually or in Excel sheets. This creates several problems such as:

- Difficulty in tracking inventory (which books are available or out of stock).
- No clear way to find top-selling books or best authors.
- Problems in analyzing customer purchase patterns (e.g., which customers buy the most or which countries prefer certain authors).
- Lack of clarity about revenue by genre or profitability of books.
- Slow and confusing report generation for decision-making.

Because of these issues, the PageTurners Bookstore cannot take timely, data-driven decisions about restocking, pricing, or marketing.



To solve these problems, this project uses **SQL Server Management Studio (SSMS)** to design a structured database system. Our purpose to solve queries through SQL is to:

- Store and manage data properly in tables.
- Run queries to extract useful information.
- Identify sales patterns like top-selling books or most profitable genres.
- Update records efficiently (e.g., increase prices by 10%).
- Generate clear reports for better business decisions.

2. Objectives of the Project

The aim of this project is to design a database for **PageTurners Bookstore** and use SQL Server to solve the owner's business problems.

To begin with, we conducted an **interview with the bookstore owner** to understand the challenges faced in managing sales, customers, books, and suppliers. From this discussion, we identified several important business questions that the owner wants to address. Additionally, during our analysis, we identified more queries that could provide valuable insights.

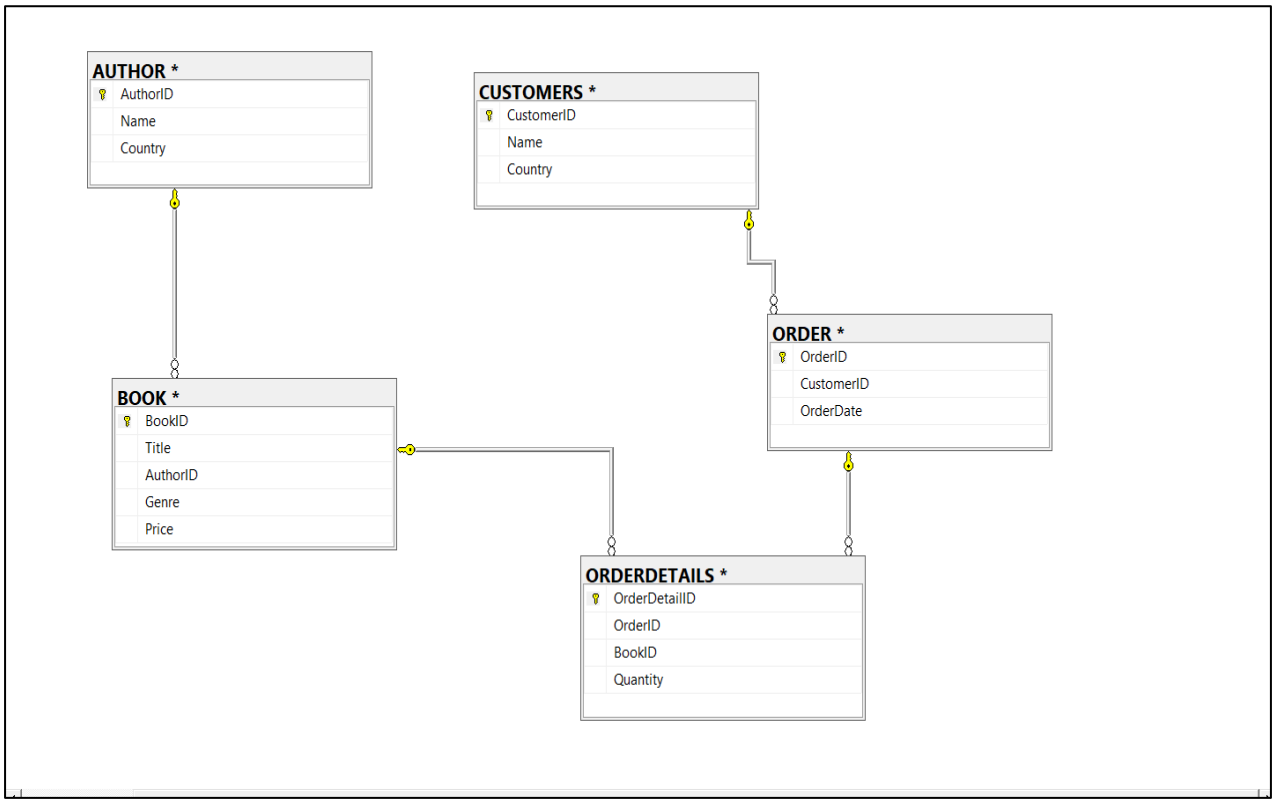
Through this project, we aim to:

- Create tables for authors, books, customers, suppliers, and orders.
- Clean the data by removing duplicate values and checking for missing (NULL) values.
- Solve the bookstore owner's key queries using SQL, such as:
    - What are all the books and their prices?
    - Which customers bought books by authors from the UK?
    - How many books are there in each genre (Fantasy, Fiction, Biography, Literary)?
    - Which books cost more than $14?
    - Which customers purchased which books?
    - What are all the books along with their authors and genres?
    - Which customers ordered books costing more than $12?
    - What are the top 3 best-selling books by quantity?
    - What is the total revenue earned per genre?
    - What is the average price of books per author?

By addressing these queries with SQL queries, the project will help the PageTurners Bookstore owner make better decisions related to **stock management, pricing strategies, marketing campaigns, and customer relationship management**.

3. Database Design

To better understand how different tables are connected in the database, we created a **Relational Diagram**. This diagram visually represents the structure of the database and shows how the entities (tables) such as authors, books, customers, orders and OrderDetails are linked with each other through relationships (primary keys and foreign keys).



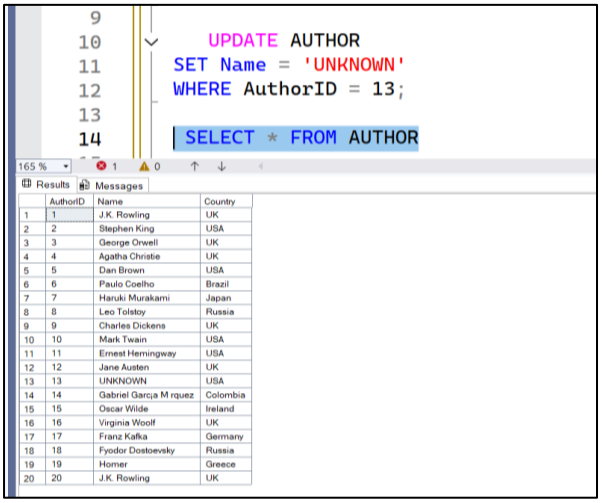
The following points show the relationships between the tables in the **Relational Diagram**:

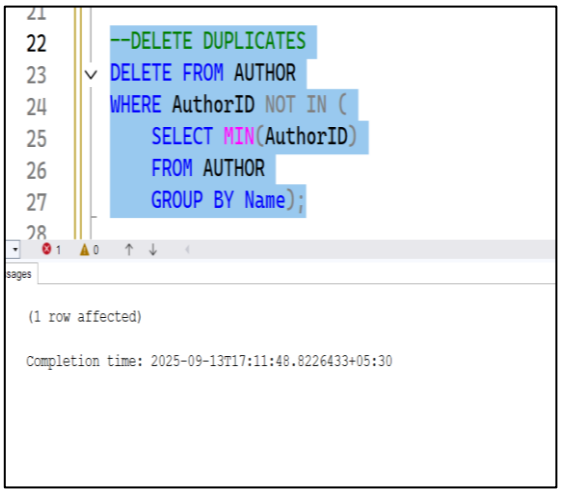
- **AUTHOR → BOOK**: Each author can write multiple books. This is a one-to-many relationship, where one AuthorID maps to many Book records.
- **CUSTOMERS → ORDER**: Each customer can place multiple orders. This is a one-to-many relationship, where one CustomerID is linked to many orders.
- **ORDER → ORDERDETAILS**: Each order can contain multiple books. The OrderDetails table acts as a bridge, storing which books and how many were purchased in each order.
- **BOOK → ORDERDETAILS**: Each book can appear in multiple orders. This shows a many-to-many relationship between Books and Orders, resolved through the OrderDetails table.

4. Methodology

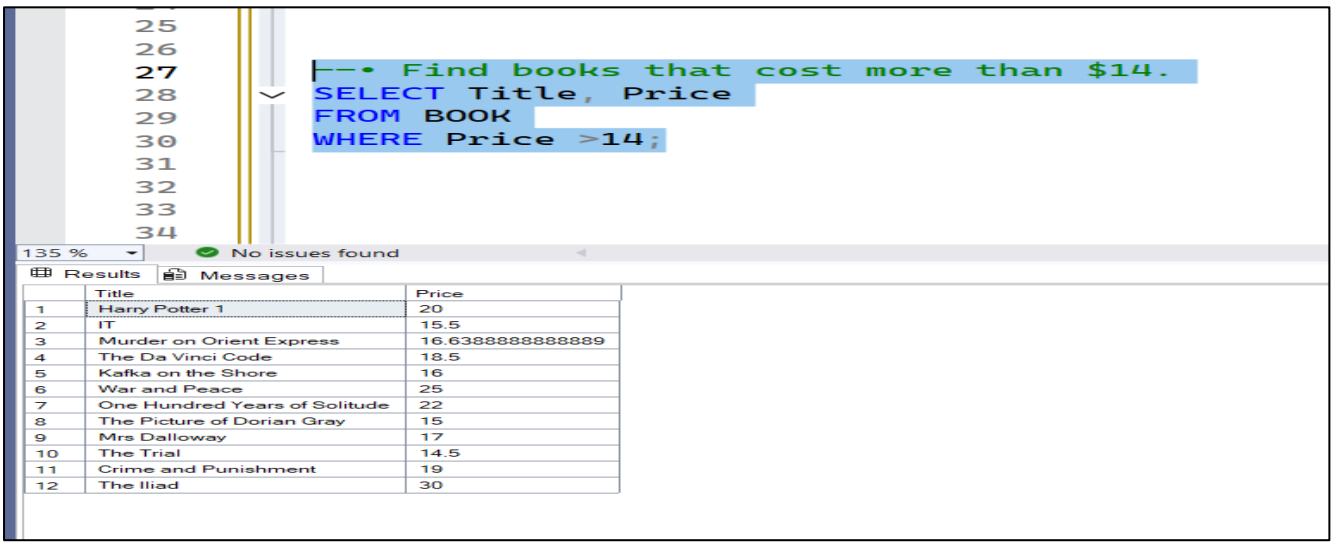
Before describing the step-by-step work, the following lists the kinds of SQL queries and features we used in the PageTurners Bookstore project, explanation of how each was applied.

DML (Data Manipulation Language) — INSERT, UPDATE, DELETE
Used to populate tables with initial/sample data, correct data errors, update missing fields, and remove duplicate or invalid rows during the cleaning phase.





DQL / SELECT queries — SELECT ... FROM ... WHERE ... (with JOIN, GROUP BY, HAVING, ORDER BY)
The core of the analysis: retrieve lists of books and prices, find customers who bought books by UK authors, compute counts per genre, filter books by price, and list which customers bought which books.



Joins & Relationship Queries — INNER JOIN, LEFT JOIN, RIGHT JOIN
Used to combine data across tables (e.g., Books ↔ Authors, Orders ↔ OrderDetails ↔ Books, Customers ↔ Orders) so we can answer cross-table business questions.

```sql
-- List customers and the titles of the books they purchased.
SELECT c.Name AS Customer, b.Title AS BOOK
FROM CUSTOMERS c
JOIN [Order] o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN BOOK b ON od.BookID = b.BookID;
```

135 % — No issues found

Results | Messages

	Customer	BOOK
1	Alice Brown	Harry Potter 1
2	Isabel Scott	Adventures of Tom Sawyer
3	Jack Turner	The Old Man and Sea
4	Karen White	Pride and Prejudice
5	Liam Hall	One Hundred Years of Solitude
6	Mary Adams	The Picture of Dorian Gray
7	Nick Young	Mrs Dalloway
8	Olivia Evans	The Trial
9	Paul Green	Crime and Punishment
10	Queen Taylor	The Iliad
11	Bob Smith	IT
12	Charlie Lee	Murder on Orient Express
13	David Johnson	The Da Vinci Code
14	Emma Wilson	The Alchemist
15	Frank Harris	Kafka on the Shore
16	Grace Kim	War and Peace
17	Henry Clark	Oliver Twist

Aggregate functions & grouping — COUNT(), SUM(), AVG(), MAX(), MIN()
Used to calculate genre-wise book counts, total revenue per genre, average price per author, and top-selling titles.

```sql
-- Get the total revenue per genre.
SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
FROM BOOK b
JOIN ORDERDETAILS od ON b.BookID = od.BookID
GROUP BY Genre;
```

135 % — No issues found

Results | Messages

	Genre	TotalRevenue
1	Epic	60
2	Fantasy	72
3	Fiction	267.5
4	Historical	25
5	Horror	15.5
6	Literary	17
7	Mystery	49.9166666666667
8	Romance	14
9	Thriller	37
10	UNKWON	29

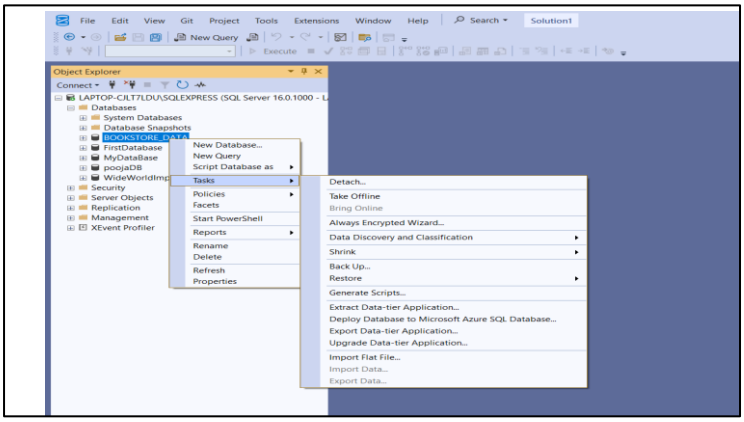
4.1 Database Creation

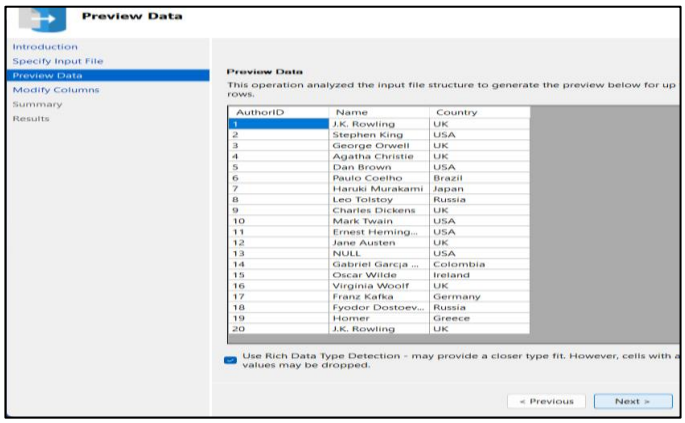
- A new database named **Bookstore** was created.
- Tables were planned for *Authors, Books, Customers, Orders, OrderDetails.*
- Each table had proper columns such as BookID, AuthorID, CustomerID, Price, and Genre.

4.2 Data Insertion (Importing Excel Data into SQL Server)

- The sample data was first prepared in **Excel sheets** (for example, Authors.xlsx, Books.xlsx, Customers. etc.).
- In SQL Server Management Studio, we clicked on the **Bookstore database**, then selected **Tasks → Import Data**.
- A new screen appeared where we chose **Excel** as the data source.

- We clicked **Browse** and selected the Excel file (for example, Authors).
- we mapped the Excel columns with SQL Server table columns and set options like **Pri0mary Key** and whether a column allows **NULL or NOT NULL** values.
- SQL Server processed the file, and finally the data was successfully inserted into the database tables.





4.3 Data Cleaning

While working on the PageTurners Bookstore database, two main issues were identified: **missing values (NULLs)** and **duplicate records**. These problems can create confusion and reduce the accuracy of reports if not handled properly.

In some cases, missing values may not have a significant impact. For example, if a customer's middle name is missing, it does not affect sales analysis or revenue reports. However, in other cases, missing values can directly influence results. For instance, if the price of a book or the quantity sold is missing, calculations of revenue or profit will be incorrect. Similarly, duplicate records may lead to inflated counts of customers, books, or sales, giving misleading insights.

Therefore, to ensure data quality and reliability, SQL queries were applied to clean the database. This included replacing or handling NULL values where they impact analysis, and removing duplicate records to maintain consistency. By doing this, the database becomes more accurate and trustworthy, ensuring that reports reflect the true performance of the bookstore.

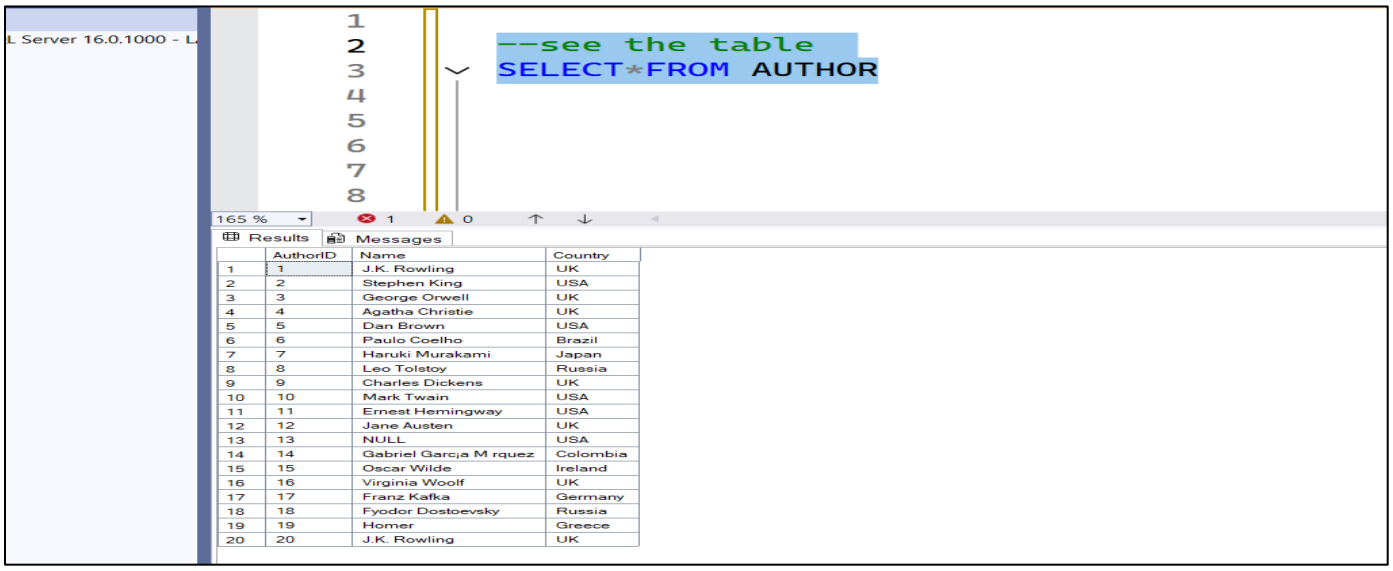
4.3.1. Handling Null Values

- Some records had missing details, such as customers without a country name or books without prices.
- To check for null values, we used the following query:

Authors Table —

- **Problem Found:** Some rows had missing or placeholder values (NULL, -) in the *Name* column. Duplicate author names also appeared.
- **Action Taken:**
  - ✓ Updated missing/invalid author names to "UNKNOWN".
  - ✓ Removed duplicate author entries, keeping only one record for each name.

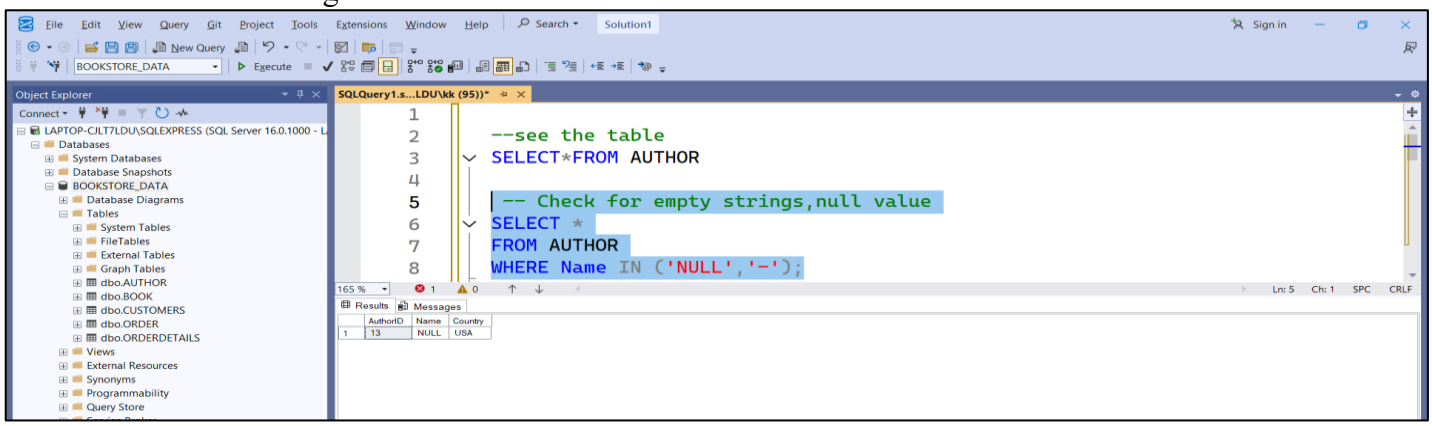
--Before Clean the Author Table view



```
--see the table
SELECT*FROM AUTHOR
```

	AuthorID	Name	Country
1	1	J.K. Rowling	UK
2	2	Stephen King	USA
3	3	George Orwell	UK
4	4	Agatha Christie	UK
5	5	Dan Brown	USA
6	6	Paulo Coelho	Brazil
7	7	Haruki Murakami	Japan
8	8	Leo Tolstoy	Russia
9	9	Charles Dickens	UK
10	10	Mark Twain	USA
11	11	Ernest Hemingway	USA
12	12	Jane Austen	UK
13	13	NULL	USA
14	14	Gabriel Garcia M rquez	Colombia
15	15	Oscar Wilde	Ireland
16	16	Virginia Woolf	UK
17	17	Franz Kafka	Germany
18	18	Fyodor Dostoevsky	Russia
19	19	Homer	Greece
20	20	J.K. Rowling	UK

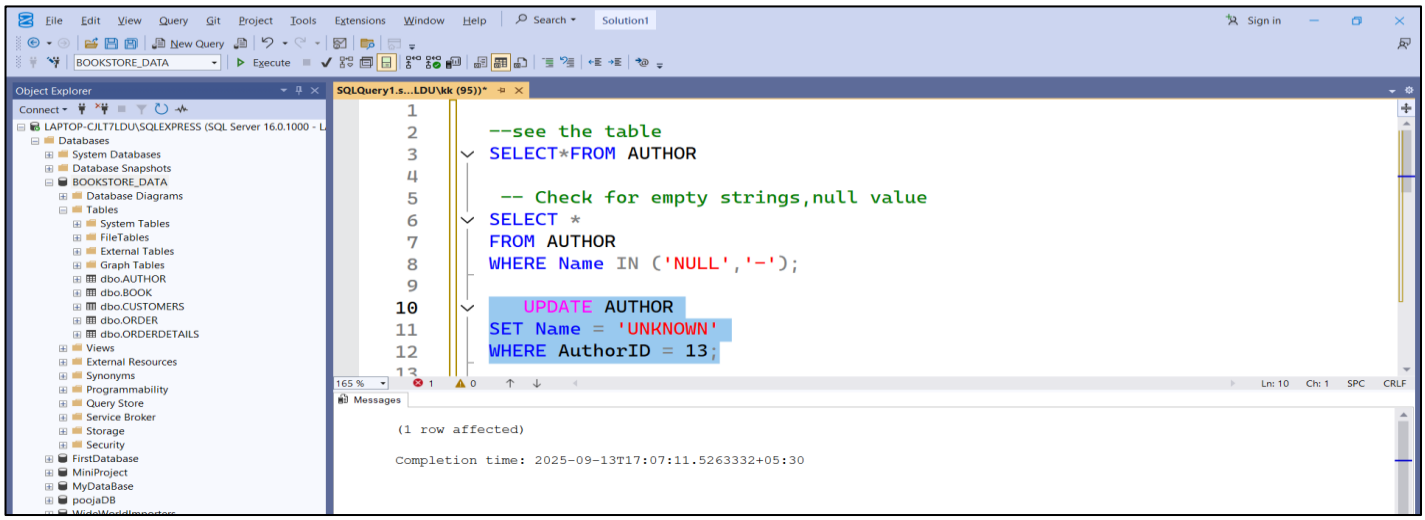
-- Find rows with missing values in Author table



```
--see the table
SELECT*FROM AUTHOR

 -- Check for empty strings,null value
SELECT *
FROM AUTHOR
WHERE Name IN ('NULL','-');
```

- Once identified, we either **updated the missing data** with default values or,
- if the row was not useful, **deleted it**.

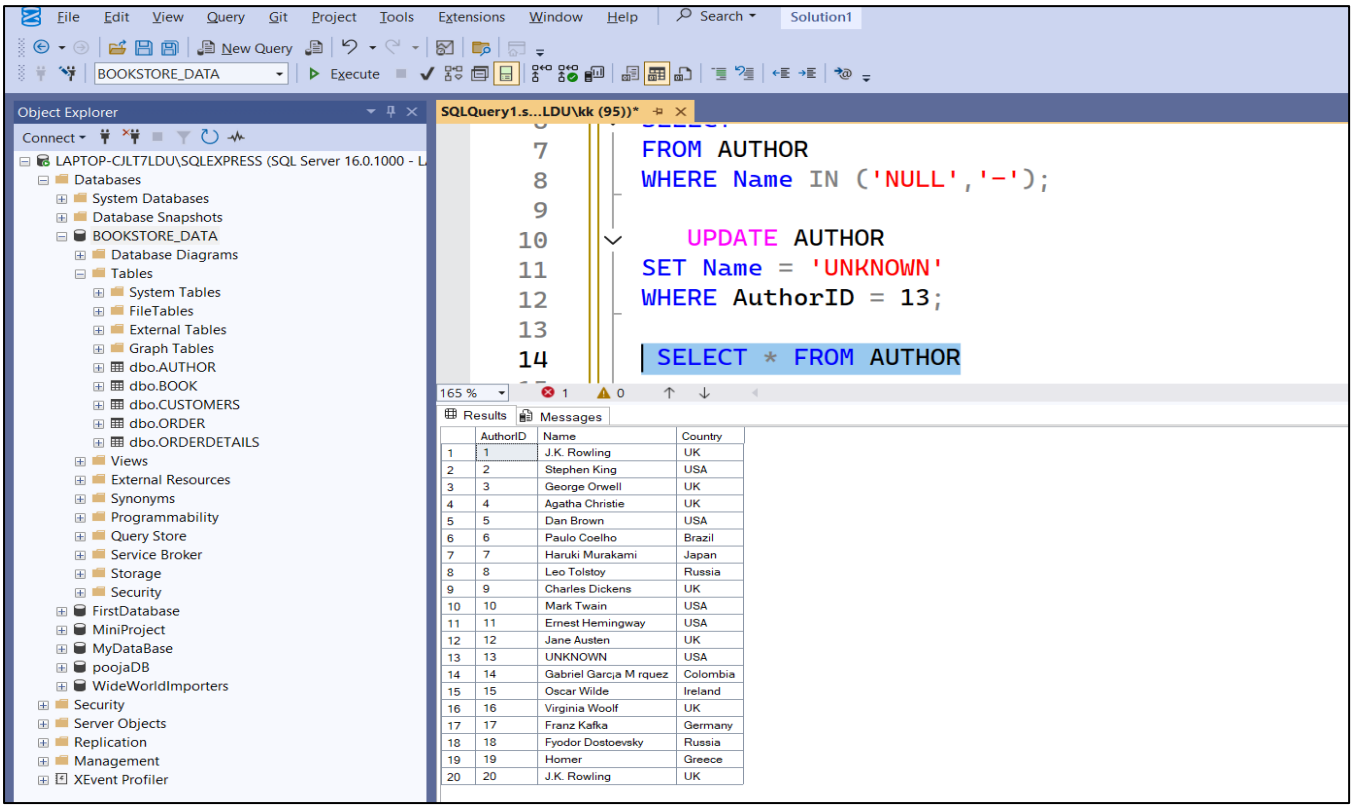
-- Updating null values:



```
--see the table
SELECT*FROM AUTHOR

 -- Check for empty strings,null value
SELECT *
FROM AUTHOR
WHERE Name IN ('NULL','-');

   UPDATE AUTHOR
SET Name = 'UNKNOWN'
WHERE AuthorID = 13;
```

```
(1 row affected)

Completion time: 2025-09-13T17:07:11.5263332+05:30
```

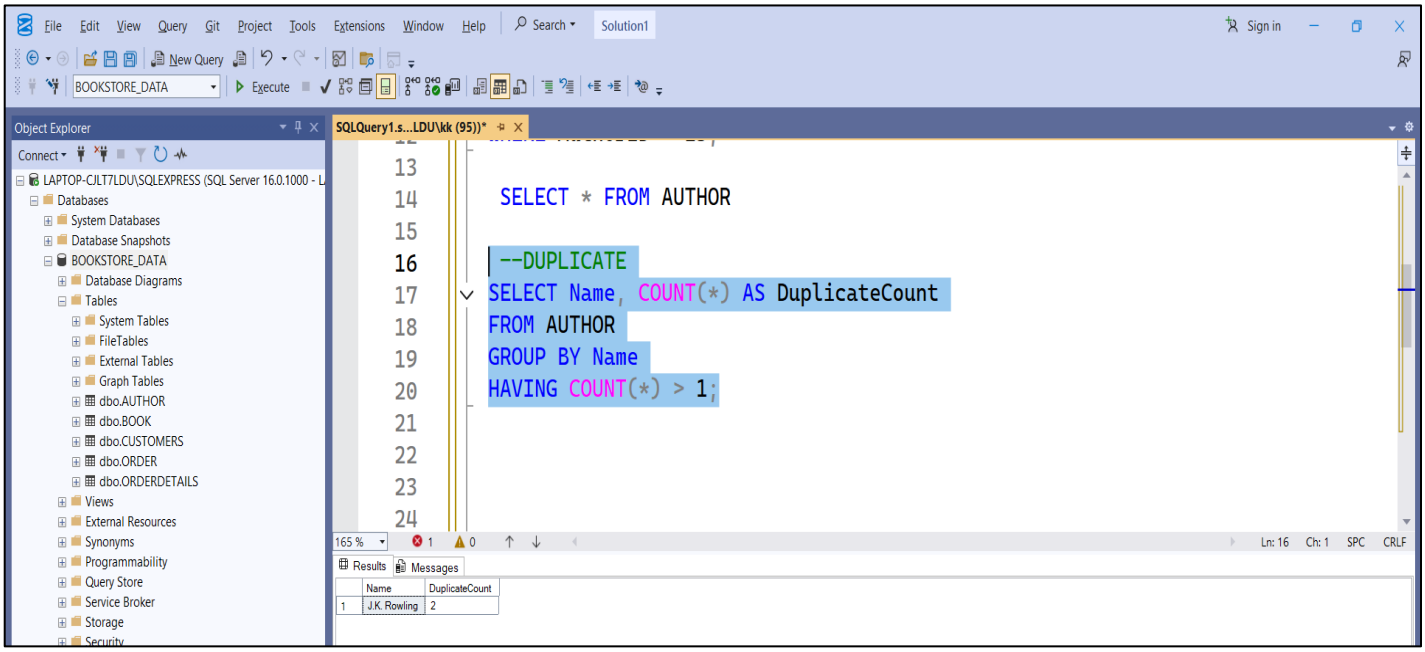
--After View Author Table



	AuthorID	Name	Country
1	1	J.K. Rowling	UK
2	2	Stephen King	USA
3	3	George Orwell	UK
4	4	Agatha Christie	UK
5	5	Dan Brown	USA
6	6	Paulo Coelho	Brazil
7	7	Haruki Murakami	Japan
8	8	Leo Tolstoy	Russia
9	9	Charles Dickens	UK
10	10	Mark Twain	USA
11	11	Ernest Hemingway	USA
12	12	Jane Austen	UK
13	13	UNKNOWN	USA
14	14	Gabriel Garcja M rquez	Colombia
15	15	Oscar Wilde	Ireland
16	16	Virginia Woolf	UK
17	17	Franz Kafka	Germany
18	18	Fyodor Dostoevsky	Russia
19	19	Homer	Greece
20	20	J.K. Rowling	UK

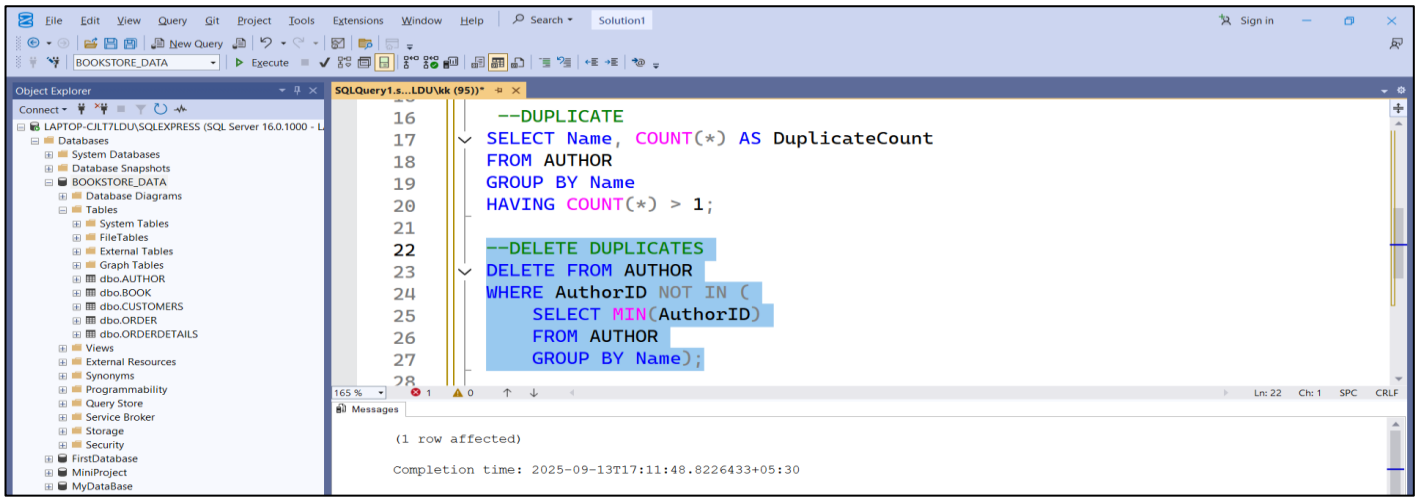
4.3.2. Removing Duplicate Records

- Some tables contained the same record more than once (e.g., the same customer repeated).
- To find duplicates, we used:
- -- Find duplicate

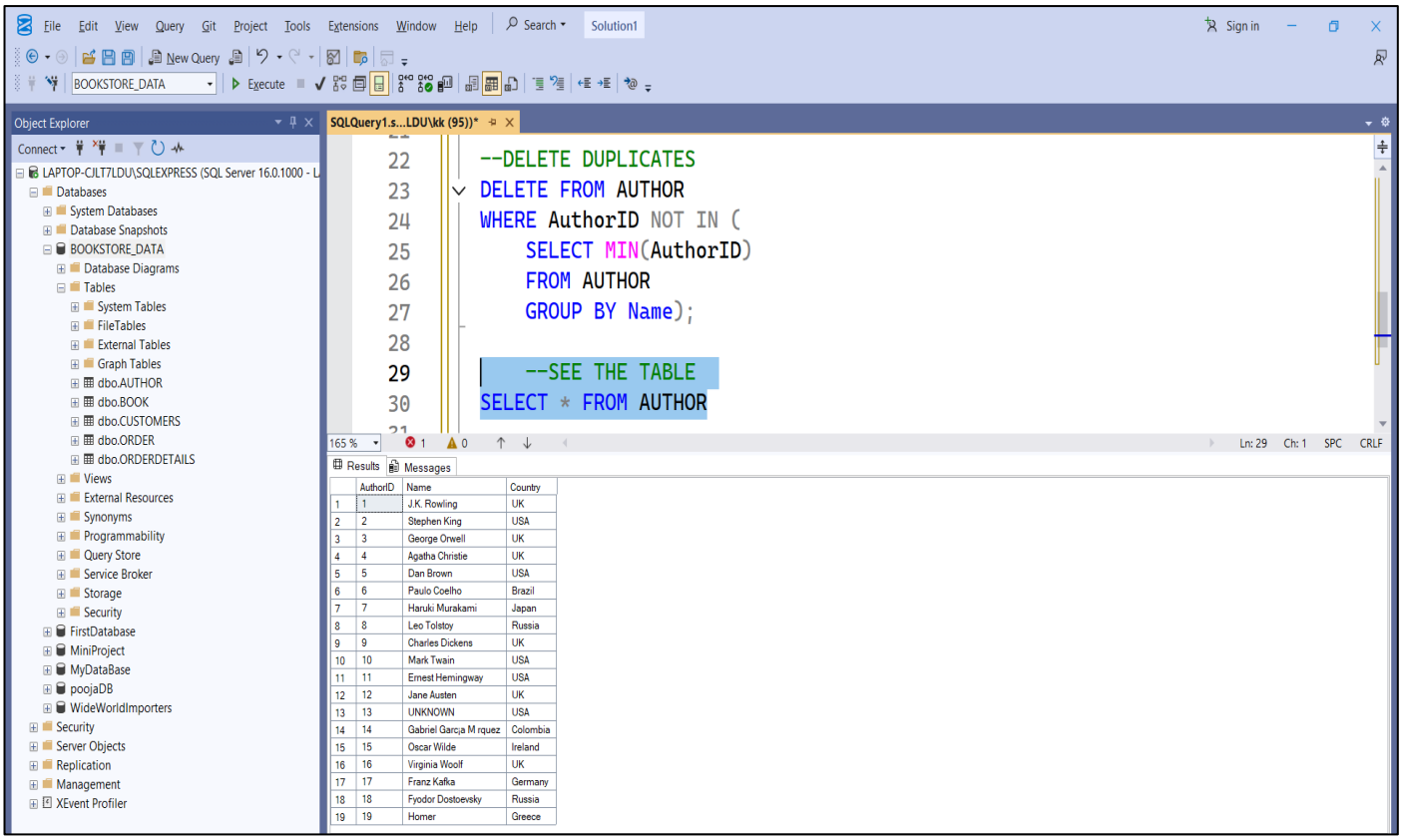


	Name	DuplicateCount
1	J.K. Rowling	2

-- Remove duplicate customer records-



After handling NULL value and Duplicate Record table look like this-



✅ **Result:**

After cleaning, the database became more accurate and reliable. Reports generated from this data are now trustworthy and help the PageTurners Bookstore owner make better business decisions..

Books Table —

- **Problem Found:** Missing or invalid values in *Title*, *Genre*, and *Price* columns. Duplicate books existed with the same *Title* and *AuthorID*.
- **Action Taken:**
  - ✓ Updated invalid or placeholder *Title* and *Genre* to "UNKNOWN".
  - ✓ Replaced missing *Price* values with the average book price.
  - ✓ Removed duplicate records, keeping the first occurrence of each book.
- **Result:** Books table now has complete and consistent values, with no duplicate records.

Customers Table —

- **Problem Found:** Missing or placeholder values in *Name* and *Country*. Duplicate customers with the same *Name* and *Country*.
- **Action Taken:**
  - ✓ Updated invalid names and countries to "UNKNOWN".
  - ✓ Removed duplicate customer entries using GROUP BY Name, Country.
- **Result:** Customers table now contains clean and unique customer records.

Orders Table —

- **Problem Found:** Some rows had missing *OrderDate*. Duplicate orders were found for the same *CustomerID*.
- **Action Taken:**
  - ✓ Updated missing *OrderDate* with a default value (11-Feb-2024).
  - ✓ Deleted duplicate orders, keeping only the earliest record for each customer.
- **Result:** Orders table now has no missing dates and contains only unique order entries.

OrderDetails Table —

- **Problem Found:** Missing values in *Quantity*. Duplicate order details for the same *OrderID*.
- **Action Taken:**
  - ✓ Updated missing quantities with the rounded average of all non-null quantities.
  - ✓ Deleted duplicate order details, keeping one record per order.
- **Result:** OrderDetails table now has valid quantities and no duplicate rows.

5. Results and Discussion

After cleaning the data, different SQL queries were executed to answer the PageTurners Bookstore owner's queries. The results are summarized below:

1. **Books and Prices Query**:

```sql
--• Select all books and their prices.
SELECT Title,Price
FROM BOOK;
```

135 % — No issues found

Results | Messages

	Title	Price
1	Harry Potter 1	20
2	IT	15.5
3	1984	13
4	Murder on Orient Express	16.6388888888889
5	The Da Vinci Code	18.5
6	The Alchemist	12
7	Kafka on the Shore	16
8	War and Peace	25
9	Oliver Twist	10
10	Adventures of Tom Sawyer	11.5
11	The Old Man and Sea	13.5
12	Pride and Prejudice	14
13	One Hundred Years of Solitude	22
14	The Picture of Dorian Gray	15
15	Mrs Dalloway	17
16	The Trial	14.5
17	Crime and Punishment	19
18	The Iliad	30
19	UNKNOWN	13

- *Insight:* Helps the owner quickly check and compare prices of all titles.

2. **Customers Who Bought Books by UK Authors**

```sql
--• Get all customers who bought books by authors from the UK.
SELECT DISTINCT c.Name
FROM CUSTOMERS c
JOIN [ORDER] O ON c.CustomerID = o.CustomerID
JOIN ORDERDETAILS od on o.[OrderID]= od.OrderID
JOIN BOOK b on od.BookID = b.BookID
Join AUTHOR a on b.AuthorID = a.AuthorID
WHERE a.Country='UK';
```

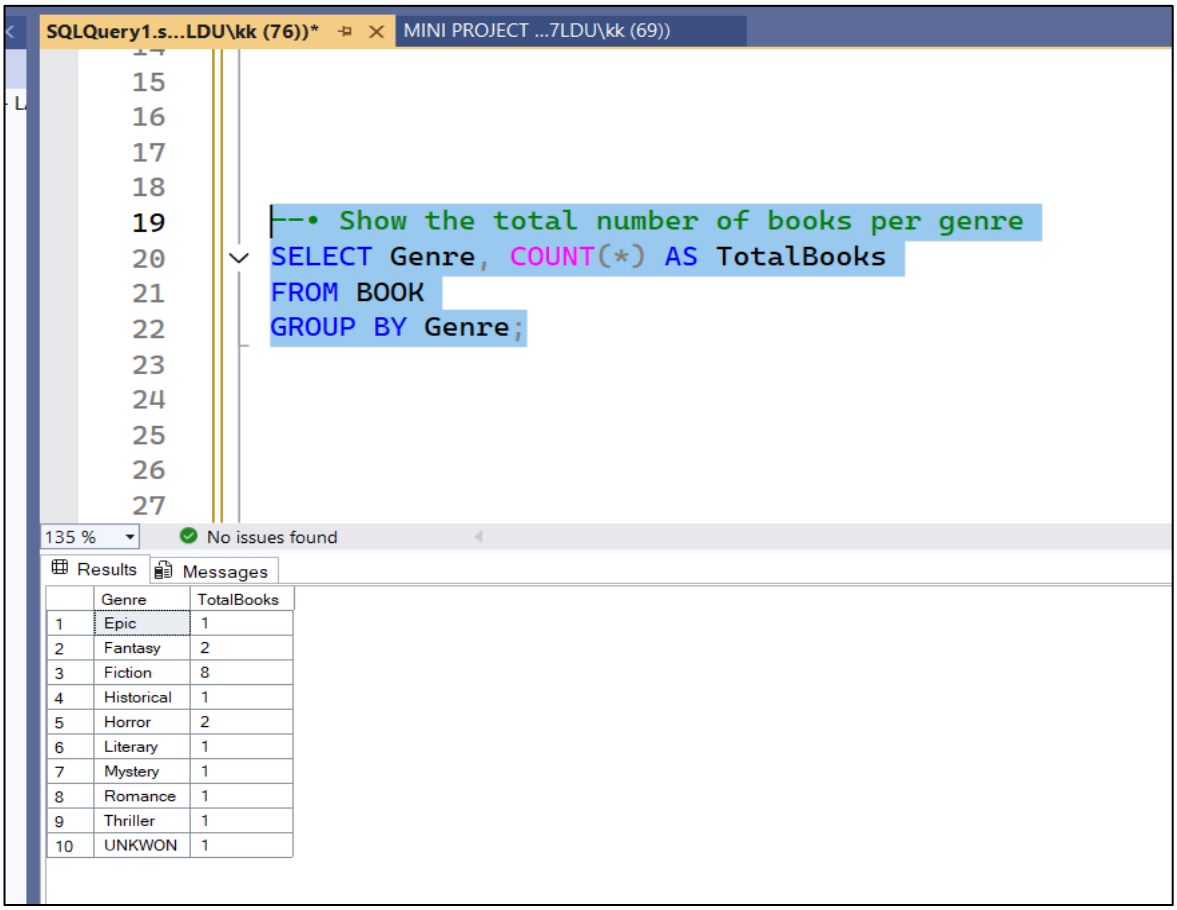
135 % — No issues found

Results | Messages

	Name
1	Alice Brown
2	Charlie Lee
3	Henry Clark
4	Karen White
5	Nick Young

- *Insight:* Useful for analyzing regional author preferences.

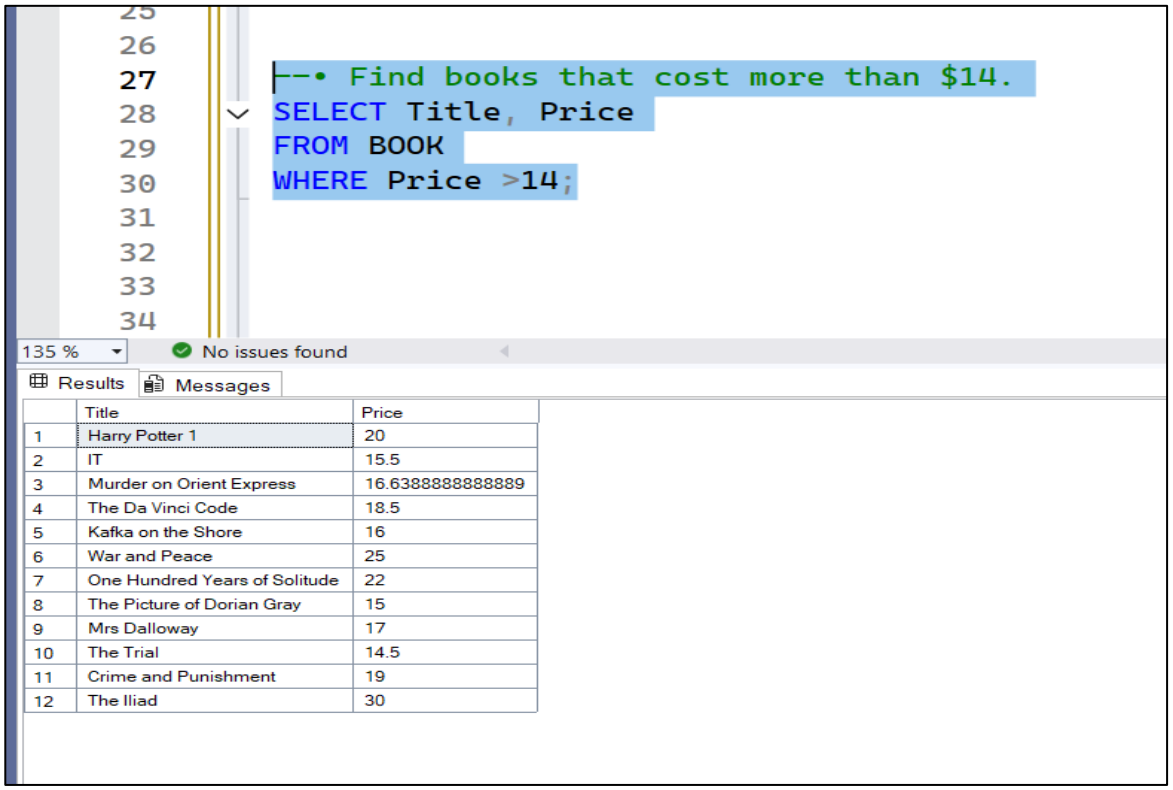
3. **Total Books per Genre**



	Genre	TotalBooks
1	Epic	1
2	Fantasy	2
3	Fiction	8
4	Historical	1
5	Horror	2
6	Literary	1
7	Mystery	1
8	Romance	1
9	Thriller	1
10	UNKWON	1

- *Insight:* Shows stock distribution across genres and highlights which categories need more inventory.

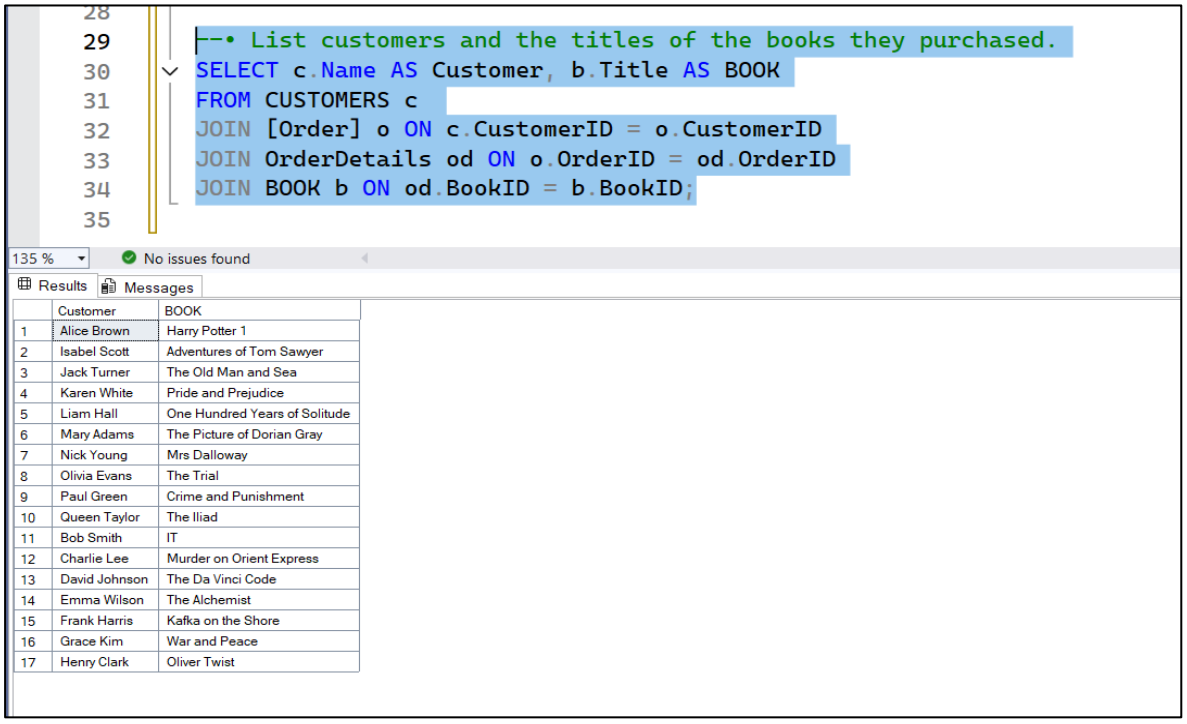
4. **Books Costing More Than $14**



	Title	Price
1	Harry Potter 1	20
2	IT	15.5
3	Murder on Orient Express	16.6388888888889
4	The Da Vinci Code	18.5
5	Kafka on the Shore	16
6	War and Peace	25
7	One Hundred Years of Solitude	22
8	The Picture of Dorian Gray	15
9	Mrs Dalloway	17
10	The Trial	14.5
11	Crime and Punishment	19
12	The Iliad	30

- *Insight:* Helps the owner manage high-value titles separately.

5. **Customers and the Books They Purchased**



```
28
29  -- List customers and the titles of the books they purchased.
30  SELECT c.Name AS Customer, b.Title AS BOOK
31  FROM CUSTOMERS c
32  JOIN [Order] o ON c.CustomerID = o.CustomerID
33  JOIN OrderDetails od ON o.OrderID = od.OrderID
34  JOIN BOOK b ON od.BookID = b.BookID;
35
```

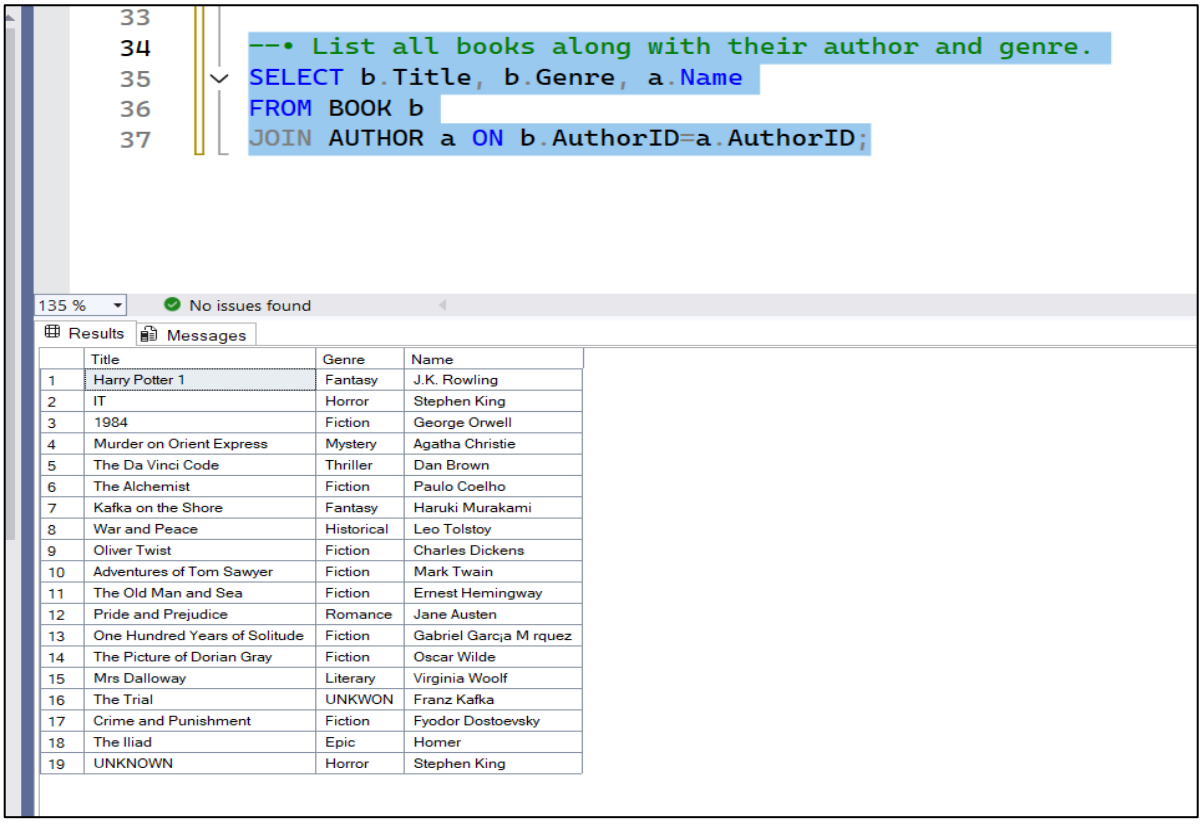
135 % — No issues found

Results | Messages

	Customer	BOOK
1	Alice Brown	Harry Potter 1
2	Isabel Scott	Adventures of Tom Sawyer
3	Jack Turner	The Old Man and Sea
4	Karen White	Pride and Prejudice
5	Liam Hall	One Hundred Years of Solitude
6	Mary Adams	The Picture of Dorian Gray
7	Nick Young	Mrs Dalloway
8	Olivia Evans	The Trial
9	Paul Green	Crime and Punishment
10	Queen Taylor	The Iliad
11	Bob Smith	IT
12	Charlie Lee	Murder on Orient Express
13	David Johnson	The Da Vinci Code
14	Emma Wilson	The Alchemist
15	Frank Harris	Kafka on the Shore
16	Grace Kim	War and Peace
17	Henry Clark	Oliver Twist

❖ *Insight:* Useful for customer management and personalized marketing.

6. **Books with Authors and Genres**



```
33
34  -- List all books along with their author and genre.
35  SELECT b.Title, b.Genre, a.Name
36  FROM BOOK b
37  JOIN AUTHOR a ON b.AuthorID=a.AuthorID;
```

135 % — No issues found

Results | Messages

	Title	Genre	Name
1	Harry Potter 1	Fantasy	J.K. Rowling
2	IT	Horror	Stephen King
3	1984	Fiction	George Orwell
4	Murder on Orient Express	Mystery	Agatha Christie
5	The Da Vinci Code	Thriller	Dan Brown
6	The Alchemist	Fiction	Paulo Coelho
7	Kafka on the Shore	Fantasy	Haruki Murakami
8	War and Peace	Historical	Leo Tolstoy
9	Oliver Twist	Fiction	Charles Dickens
10	Adventures of Tom Sawyer	Fiction	Mark Twain
11	The Old Man and Sea	Fiction	Ernest Hemingway
12	Pride and Prejudice	Romance	Jane Austen
13	One Hundred Years of Solitude	Fiction	Gabriel Garcia M rquez
14	The Picture of Dorian Gray	Fiction	Oscar Wilde
15	Mrs Dalloway	Literary	Virginia Woolf
16	The Trial	UNKWON	Franz Kafka
17	Crime and Punishment	Fiction	Fyodor Dostoevsky
18	The Iliad	Epic	Homer
19	UNKNOWN	Horror	Stephen King

❖ *Insight:* Provides a complete catalog view for reporting.

7. **Customers Who Ordered Books Above $12**

```sql
-- Find customers who ordered books costing more than $12.
SELECT DISTINCT c.Name
FROM CUSTOMERS c
JOIN [ORDER] o ON c.CustomerID= o.customerID
JOIN [ORDERDETAILS] od ON o.OrderID=od.OrderID
JOIN BOOK b ON od.BookID=b.BookID
WHERE Price>12;
```

135 % No issues found

Results Messages

	Name
1	Alice Brown
2	Bob Smith
3	Charlie Lee
4	David Johnson
5	Frank Harris
6	Grace Kim
7	Jack Turner
8	Karen White
9	Liam Hall
10	Mary Adams
11	Nick Young
12	Olivia Evans
13	Paul Green
14	Queen Taylor

❖ *Insight:* Helps identify premium customers.

8. **Average Price per Author**

```sql
-- Find the average price per author.
SELECT a.Name AS Author, AVG(b.Price) AS AvgPrice
FROM AUTHOR a
JOIN BOOK b ON a.AuthorID = b.AuthorID
GROUP BY a.Name;
```

134 % No issues found

Results Messages

	Author	AvgPrice
1	Agatha Christie	16.6388888888889
2	Charles Dickens	10
3	Dan Brown	18.5
4	Ernest Hemingway	13.5
5	Franz Kafka	14.5
6	Fyodor Dostoevsky	19
7	Gabriel Garcia M rquez	22
8	George Orwell	13
9	Haruki Murakami	16
10	Homer	30
11	J.K. Rowling	20
12	Jane Austen	14
13	Leo Tolstoy	25
14	Mark Twain	11.5
15	Oscar Wilde	15
16	Paulo Coelho	12
17	Stephen King	14.25
18	Virginia Woolf	17

❖ *Insight:* Helps in price comparison between authors.

9. **Top 3 Best-Selling Books**

```sql
-- Find the top 3 best-selling books by quantity.
SELECT TOP 3 b.Title,SUM(od.Quantity) AS TotalSales
FROM [ORDERDETAILS] od
JOIN BOOK b ON od.BookID=b.BookID
JOIN AUTHOR a ON  b.AuthorID=a.AuthorID
GROUP BY b.Title
ORDER BY TotalSales DESC
```

135 % No issues found

Results Messages

	Title	TotalSales
1	Oliver Twist	5
2	The Alchemist	4
3	Adventures of Tom Sawyer	3

❖ *Insight:* Helps plan restocking and promotions for popular titles.

10. **Total Revenue per Genre**

```
53  --• Get the total revenue per genre.
54  SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
55  FROM BOOK b
56  JOIN ORDERDETAILS od ON b.BookID = od.BookID
57  GROUP BY Genre;
```

134 % — No issues found

Results | Messages

	Genre	TotalRevenue
1	Epic	60
2	Fantasy	72
3	Fiction	267.5
4	Historical	25
5	Horror	15.5
6	Literary	17
7	Mystery	49.9166666666667
8	Romance	14
9	Thriller	37
10	UNKWON	29

❖ *Insight:* Shows which genres are most profitable for the store.

❖ **Discussion Summary**

The results proved that SQL queries are powerful tools for solving real PageTurners Bookstore problems. The owner can now:

- Track inventory and pricing.
- Identify best-selling books and profitable genres.
- Understand customer buying patterns.
- Manage suppliers effectively.
- Update records with ease.

6.Conclusion

This project showed how SQL Server can help manage PageTurners Bookstore data in an organized way. By cleaning the data and fixing errors like missing or duplicate information, the database became accurate and reliable.

Using SQL queries, we were able to answer important business questions, such as which books sold the most, how much revenue each genre made, and who the best customers were. These insights can help store managers make better decisions about stock, sales, and customer services.

Overall, this project proved that SQL is not just for storing data. It is also a powerful tool to understand business trends, solve real problems, and make smarter decisions.

7.Appendix

This appendix contains the complete SQL code, table structures, data cleaning queries, and query r used in the project. It serves as technical documentation to support the main report.

A. Database Schema (Table Structures)

Author		
Columns Name	Data Type	Other info.
AuthorID	tinyint	Primary Key Not NULL
Name	Nvarchar(50)	NULL
Country	Nvarchar(50)	Not NULL

BOOK		
Columns Name	Data Type	Other info.
BookID	tinyint	Primary Key Not NULL
Title	Nvarchar(50)	NULL
AuthorID	tinyint	Not NULL
Genre	Nvarchar(50)	NULL
Price	Float	NULL

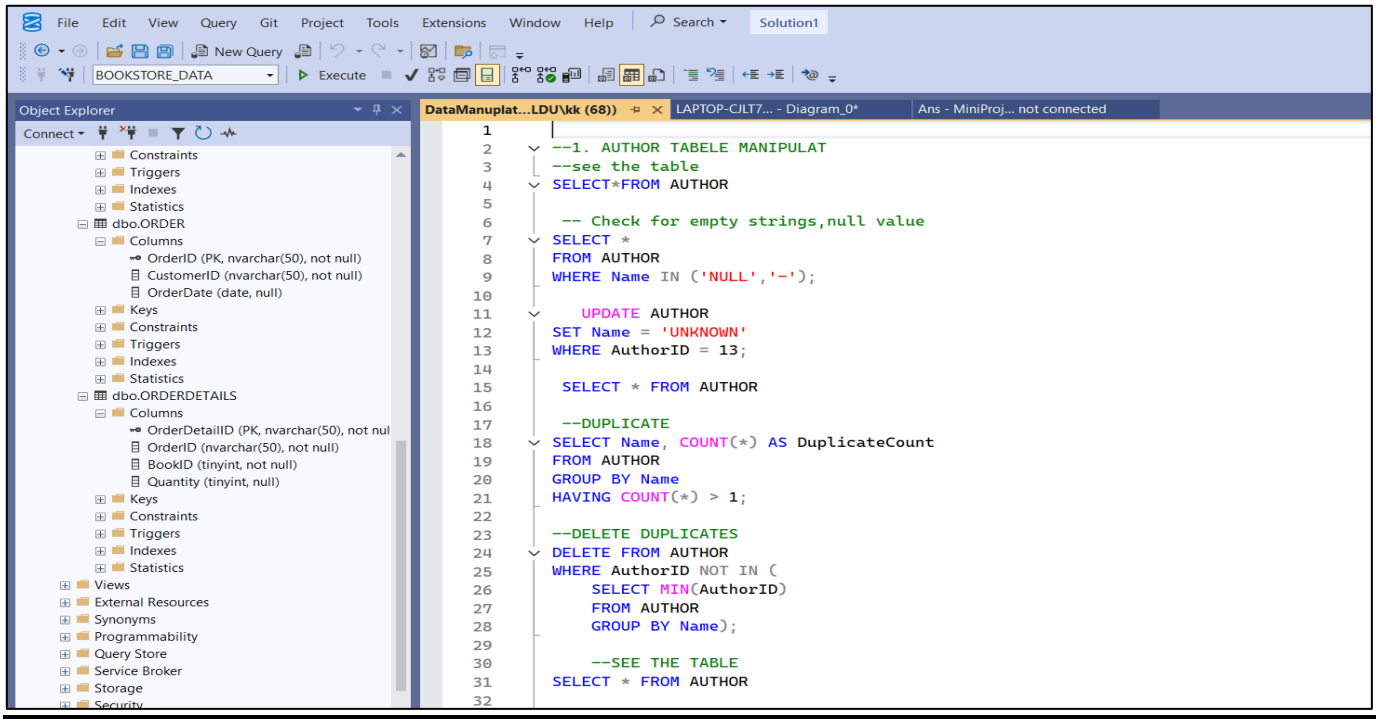
Customer		
Columns Name	Data Type	Other info.
CustomerID	tinyint	Primary Key Not NULL
Name	Nvarchar(50)	NULL
Country	Nvarchar(50)	NULL

Order		
Columns Name	Data Type	Other info.
OrderID	tinyint	Primary Key Not NULL
CustomerID	tinyint	NOT NULL
OrderDate	Date	NULL

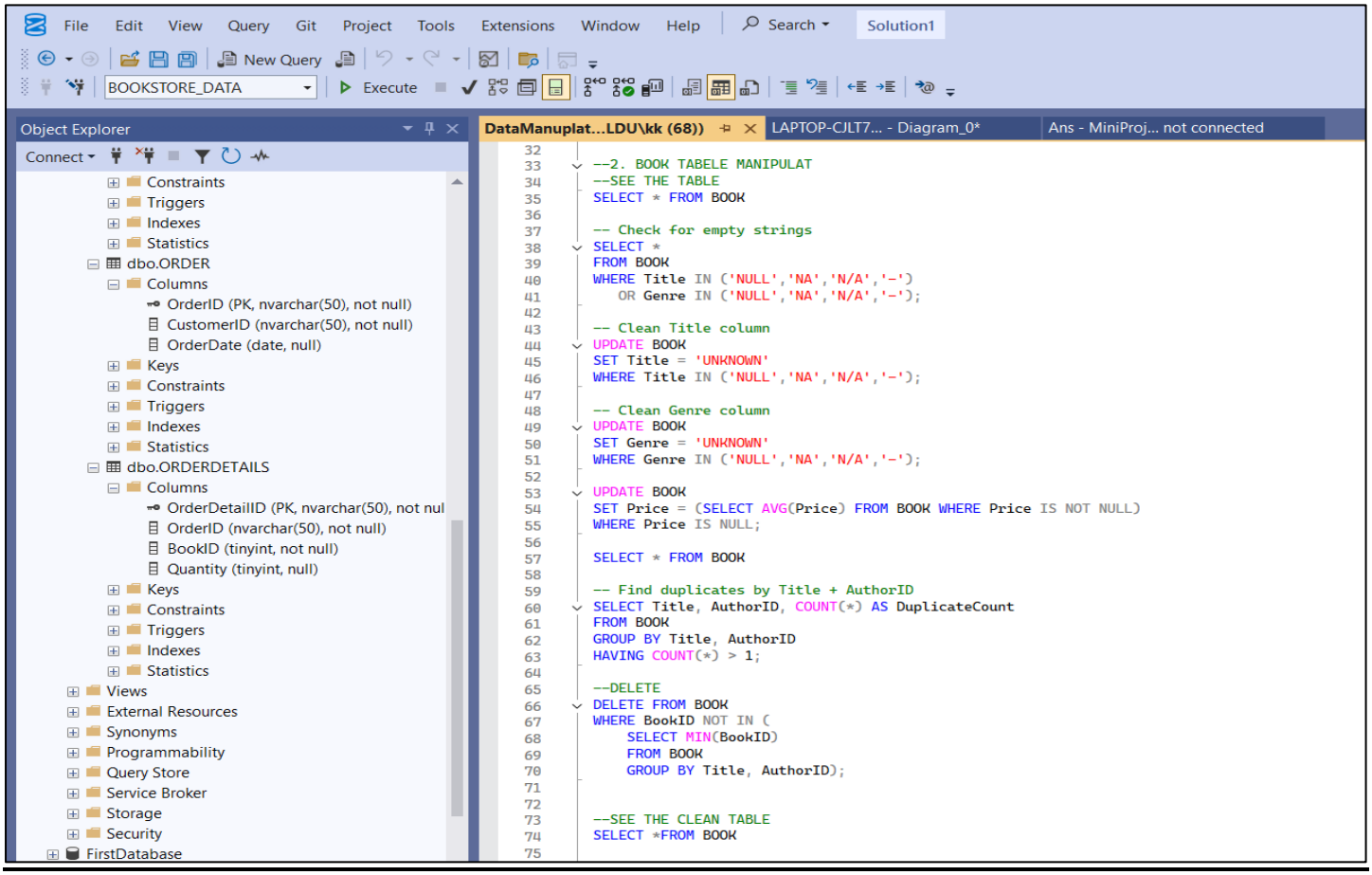
OrderDetails		
Columns Name	Data Type	Other info.
OrderDetailsID	tinyint	Primary Key Not NULL
BookID	tinyint	NULL
OrderID	tinyint	NULL
Quantity	tinyint	NOT NULL

B. Data Cleaning Queries

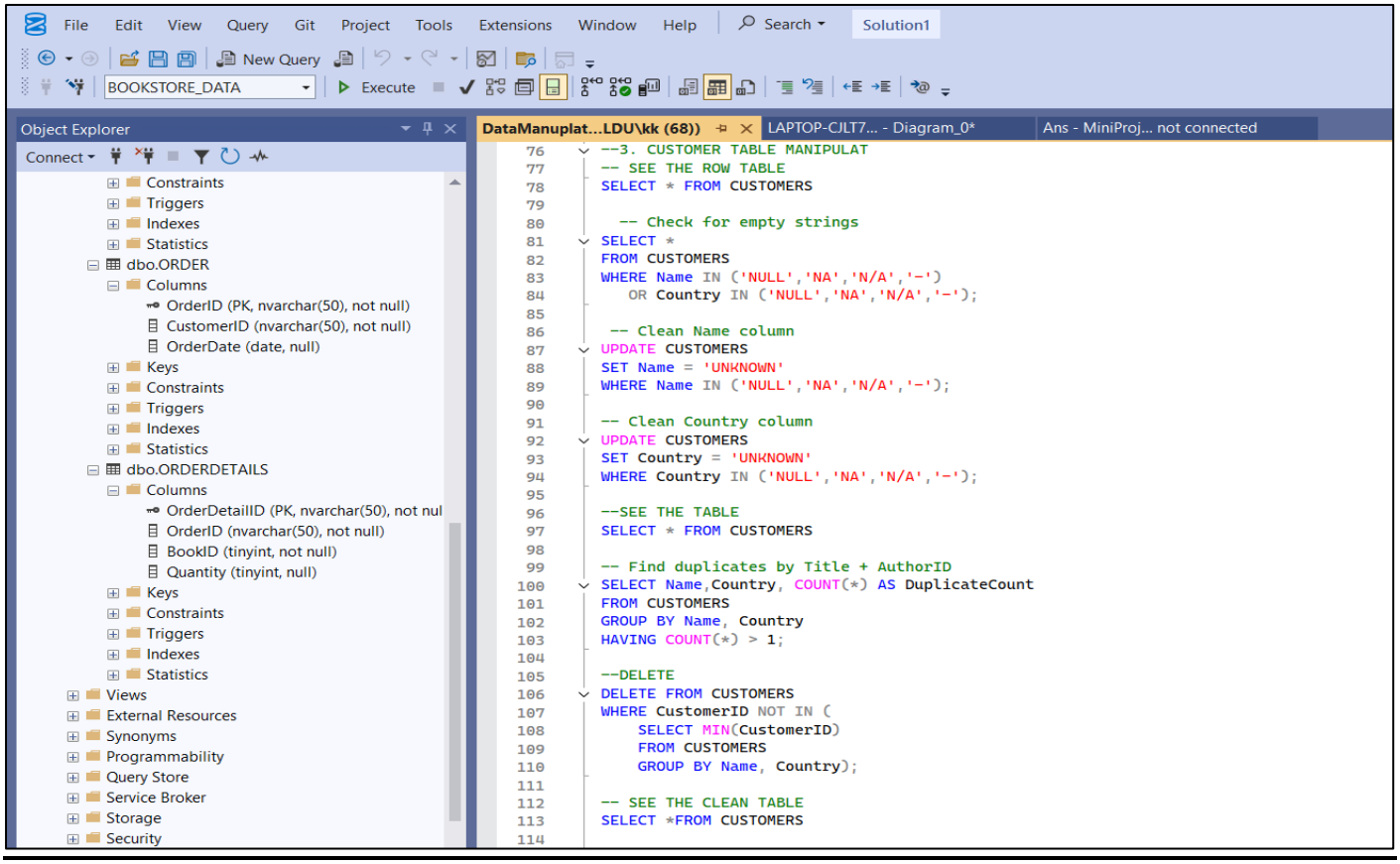
Author



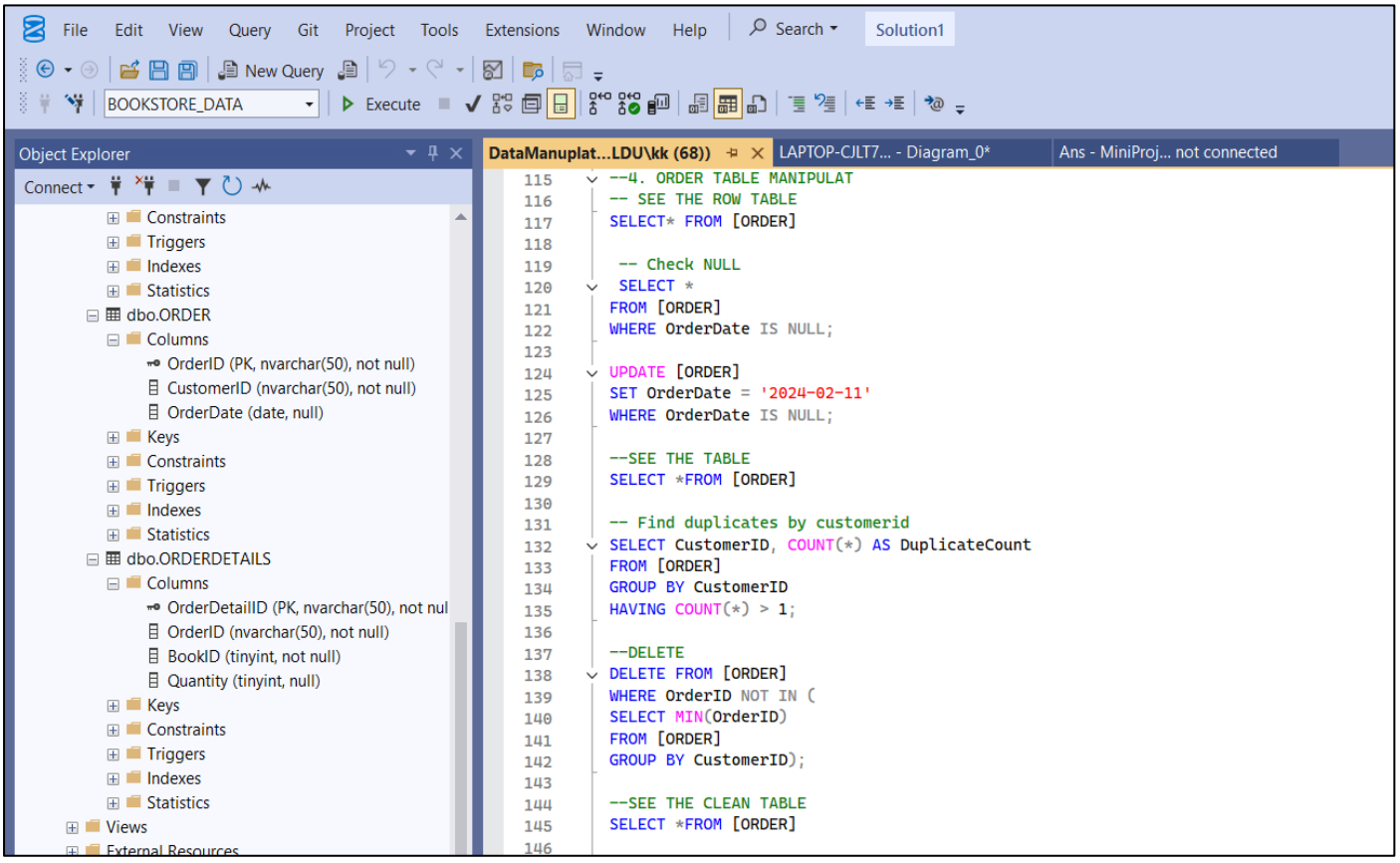
Book



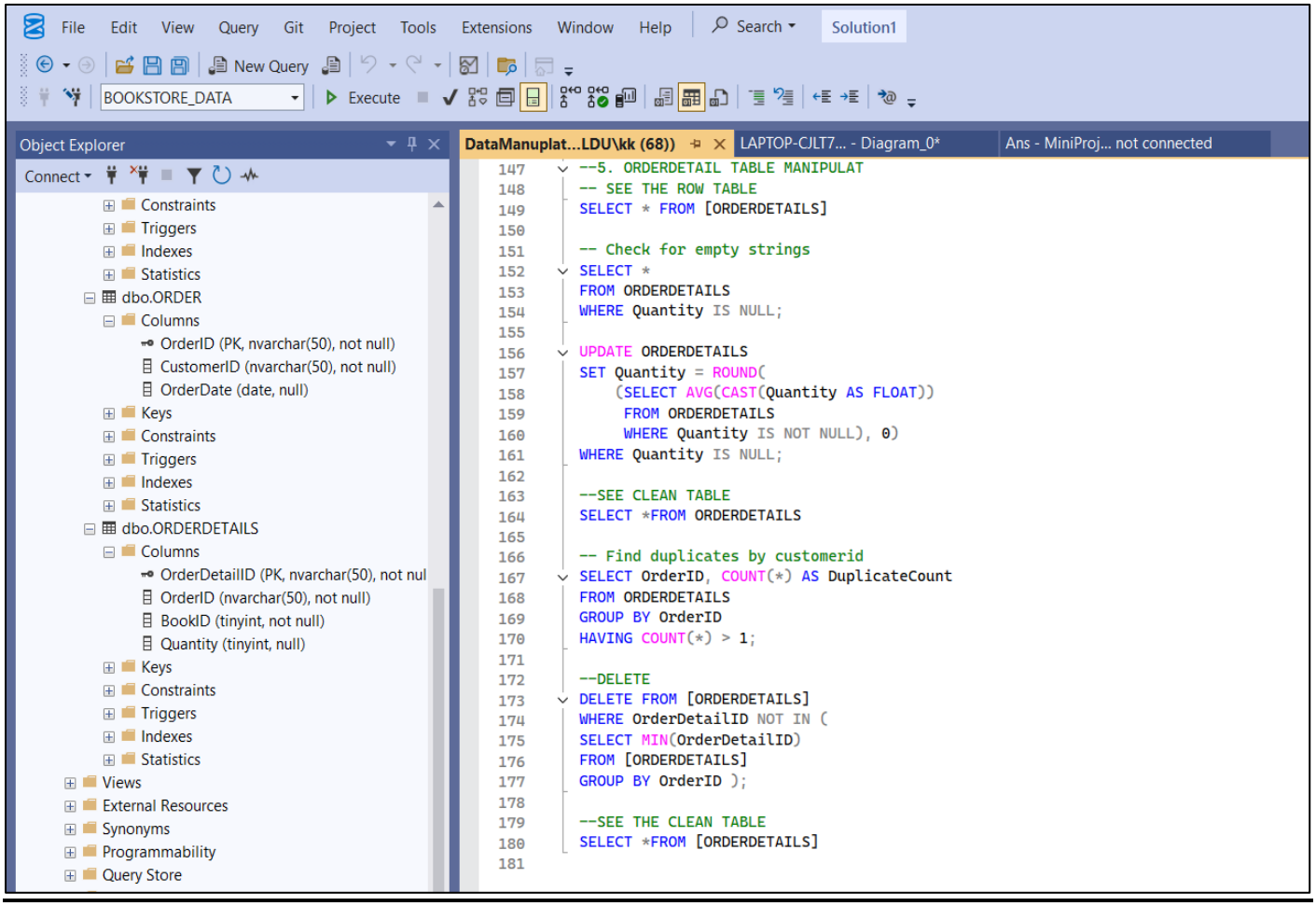
Customer



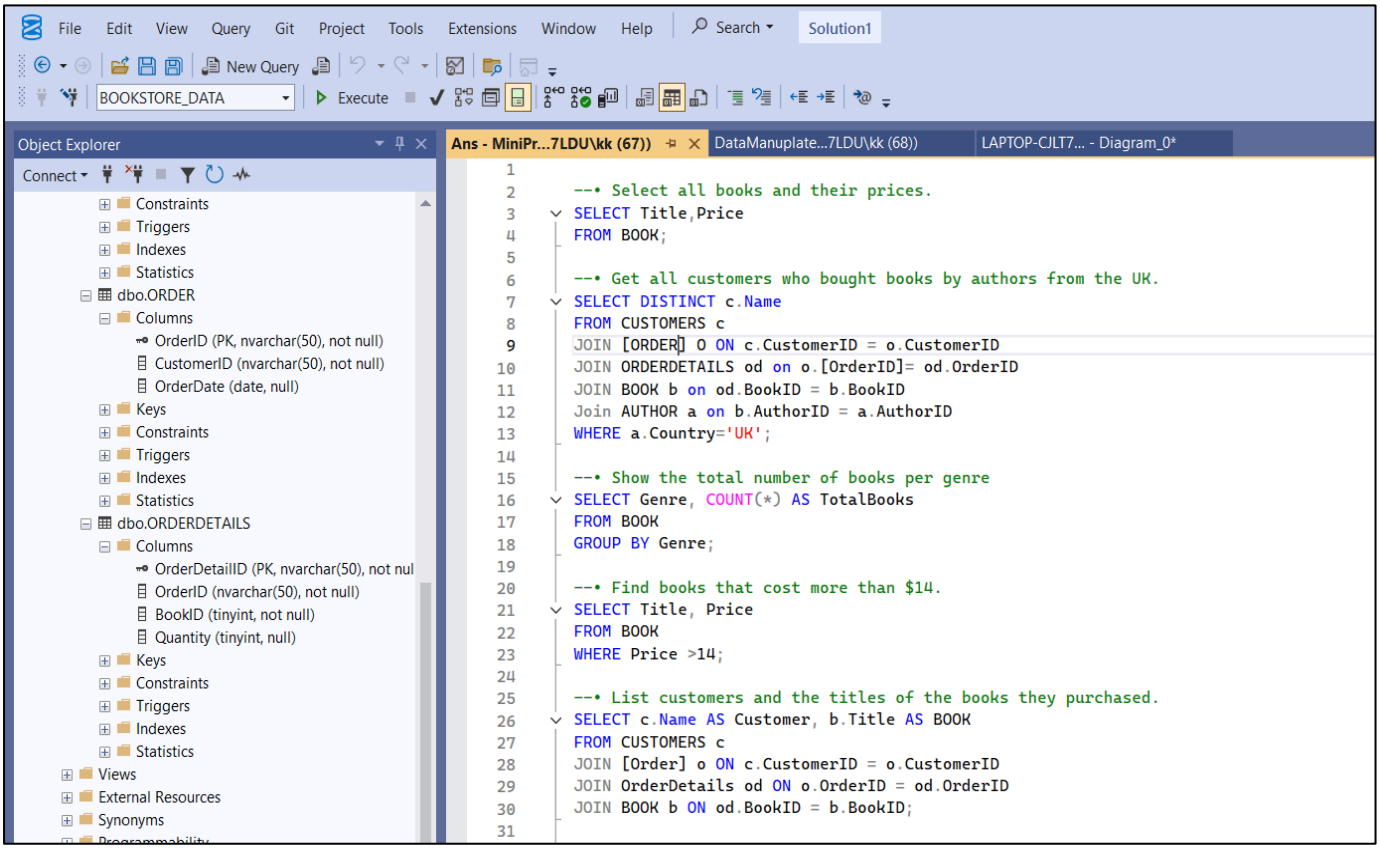
Order



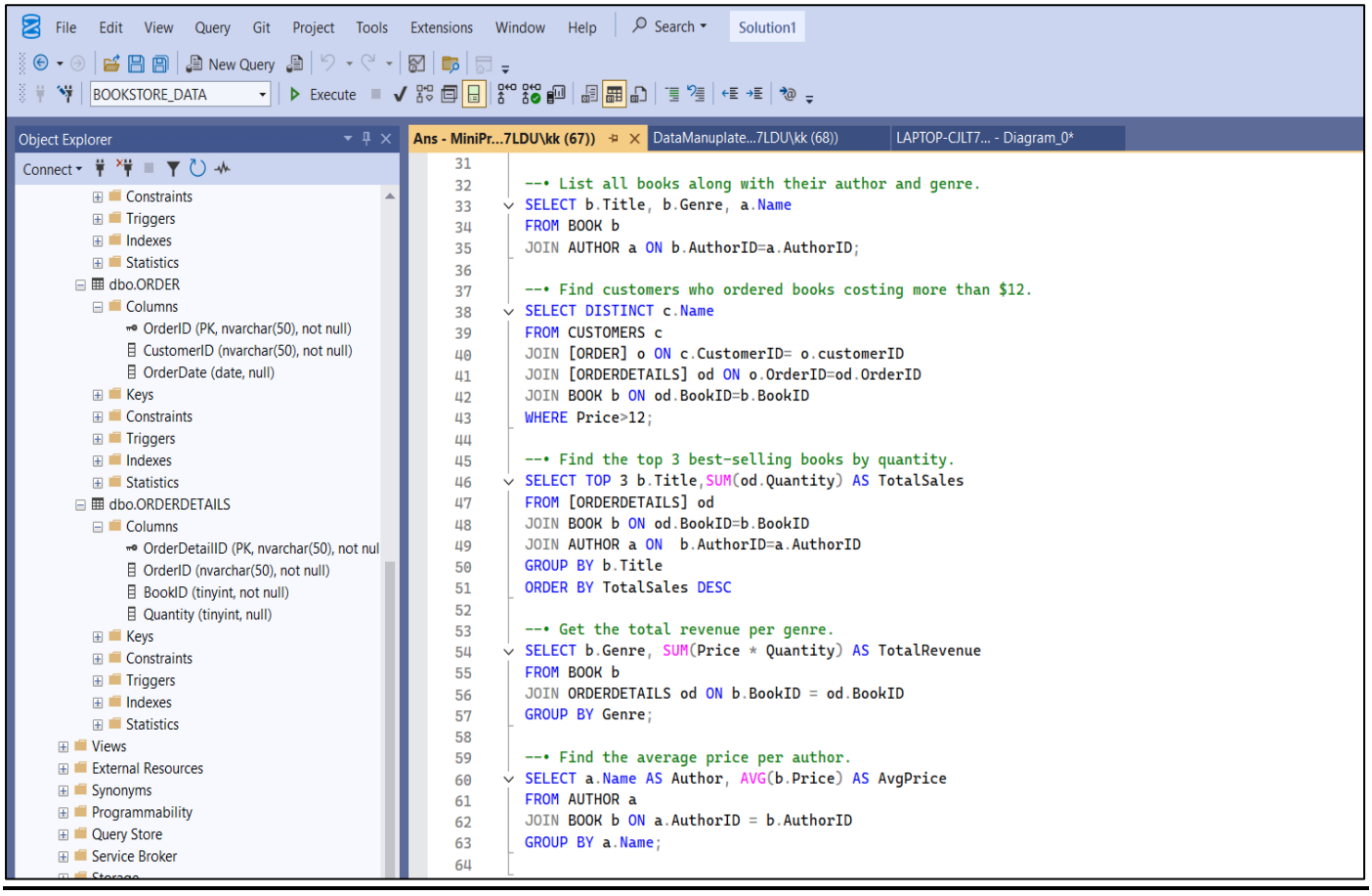
OrderDetails



C. SQL Queries:



```sql
--• Select all books and their prices.
SELECT Title,Price
FROM BOOK;

--• Get all customers who bought books by authors from the UK.
SELECT DISTINCT c.Name
FROM CUSTOMERS c
JOIN [ORDER] O ON c.CustomerID = o.CustomerID
JOIN ORDERDETAILS od on o.[OrderID]= od.OrderID
JOIN BOOK b on od.BookID = b.BookID
Join AUTHOR a on b.AuthorID = a.AuthorID
WHERE a.Country='UK';

--• Show the total number of books per genre
SELECT Genre, COUNT(*) AS TotalBooks
FROM BOOK
GROUP BY Genre;

--• Find books that cost more than $14.
SELECT Title, Price
FROM BOOK
WHERE Price >14;

--• List customers and the titles of the books they purchased.
SELECT c.Name AS Customer, b.Title AS BOOK
FROM CUSTOMERS c
JOIN [Order] o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN BOOK b ON od.BookID = b.BookID;
```



```sql
--• List all books along with their author and genre.
SELECT b.Title, b.Genre, a.Name
FROM BOOK b
JOIN AUTHOR a ON b.AuthorID=a.AuthorID;

--• Find customers who ordered books costing more than $12.
SELECT DISTINCT c.Name
FROM CUSTOMERS c
JOIN [ORDER] o ON c.CustomerID= o.customerID
JOIN [ORDERDETAILS] od ON o.OrderID=od.OrderID
JOIN BOOK b ON od.BookID=b.BookID
WHERE Price>12;

--• Find the top 3 best-selling books by quantity.
SELECT TOP 3 b.Title,SUM(od.Quantity) AS TotalSales
FROM [ORDERDETAILS] od
JOIN BOOK b ON od.BookID=b.BookID
JOIN AUTHOR a ON  b.AuthorID=a.AuthorID
GROUP BY b.Title
ORDER BY TotalSales DESC

--• Get the total revenue per genre.
SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
FROM BOOK b
JOIN ORDERDETAILS od ON b.BookID = od.BookID
GROUP BY Genre;

--• Find the average price per author.
SELECT a.Name AS Author, AVG(b.Price) AS AvgPrice
FROM AUTHOR a
JOIN BOOK b ON a.AuthorID = b.AuthorID
GROUP BY a.Name;
```

8.References

The following sources were referred to while preparing this project. They provided valuable theoretical concepts, practical examples, and guidance in understanding SQL Server and database management.

1. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th Edition). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th Edition). McGraw Hill.
3. Groff, J. R., & Weinberg, P. N. (2010). *SQL: The Complete Reference*. McGraw Hill.
4. Coronel, C., & Morris, S. (2018). *Database Systems: Design, Implementation, and Management*. Cengage Learning.